

Recursion and Time Complexity - Answers

1. Recursive Summation

```
int sum(int n){  
    if(n == 1) return 1;  
    return n + sum(n-1);  
}
```

2. Recursive Multiplication Up to Given Number (Factorial Style)

```
int multiplyUpTo(int n){  
    if(n == 1) return 1;  
    return n * multiplyUpTo(n-1);  
}
```

3. Recursive Function to Multiply Two Numbers

```
int multiply(int a, int b){  
    if(b == 0) return 0;  
    return a + multiply(a, b-1);  
}
```

4. Recursive Fibonacci Function

```
int fibonacci(int n){  
    if(n == 0) return 0;  
    if(n == 1) return 1;  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```

5. Recursive Factorial Function

```
int factorial(int n){  
    if(n == 0 || n == 1) return 1;  
    return n * factorial(n-1);  
}
```

6. Recursive Palindrome Check

```
boolean isPalindrome(String str){  
    if(str.length() <= 1) return true;  
    if(str.charAt(0) != str.charAt(str.length()-1)) return false;  
    return isPalindrome(str.substring(1, str.length()-1));  
}
```

Time Complexity Answers

1. Single loop running n times $\rightarrow O(n)$
2. Two nested loops, each running n times $\rightarrow O(n^2)$
3. n is divided by 2 each iteration $\rightarrow O(\log n)$
4. First part $O(n^2)$ + second part $O(n) = O(n^2 + n) \rightarrow O(n^2)$
5. Three nested loops, each running n times $\rightarrow O(n^3)$
6. First nested loops = $O(n^2)$
Second loop doubles each time = $O(\log n)$
Total = $O(n^2 + \log n) \rightarrow O(n^2)$